Chapter 1: Executive Summary

This report presents a comprehensive audit of the Voice JARVIS Android application, a voice-activated AI assistant designed to operate as a foreground service with a floating overlay HUD. The project follows a three-pillar architecture plan: (1) a local wake-word detection system, (2) a continuous voice conversation system powered by Google Gemini Live API via WebSocket, and (3) a proactive intelligence system that learns user habits. After a thorough line-by-line analysis of every source file, asset, build configuration, and architectural document, this report identifies a total of 41 distinct issues spanning critical failures, functional bugs, fake or misleading claims, stub code, hardcoded values, missing implementations, and architectural gaps.

The most severe finding is that the wake-word system, which is the core activation mechanism for the entire application, is fundamentally non-functional. The project claims to use a custom "Hey Jarvis" TFLite model, but the actual asset included is a generic speech commands model (speech_commands.tflite) that does not contain any "jarvis" labels. This means the wake word will never trigger, rendering the primary user interaction path completely broken. Additionally, the entire third pillar (Proactive Intelligence System) including Room database, WorkManager scheduling, and UsageStatsManager integration is entirely missing from the codebase despite being prominently featured in the project plan.

The voice conversation system (Gemini Live WebSocket client) is the most functional component and demonstrates good engineering practices including reconnection logic with exponential backoff, audio buffering with bounded channels, and proper state management. However, it lacks critical features specified in the plan such as Acoustic Echo Cancellation (AEC) and Voice Activity Detection (VAD), and has several concurrency and resource management bugs. The report below provides a detailed breakdown of every issue found, categorized by severity, along with specific code-level fix recommendations.

Severity	Count	Categories
CRITICAL	5	Wake-word model mismatch, fake documentation, state desync, entire Pillar 3 missing
HIGH	12	Resource leaks, race conditions, AEC/VAD missing, thread safety, audio drop, API key exposure, state bugs
MEDIUM	16	Hardcoded values, dead code, unused deps, empty ProGuard, no minification, lint disabled, misleading UI
LOW	8	Unused data class, stub tests, empty directories, emoji in UI, permission UX, version issues

Chapter 2: Wake Word System Analysis

The wake word system is the primary activation mechanism for the JARVIS assistant. It is designed to run continuously in a foreground service, listening for the phrase "Hey Jarvis" using a local TensorFlow Lite audio classification model. This chapter examines the WakeWordEngine, AudioController, and their integration

within the JarvisService to determine whether the system is properly implemented and correctly wired together.

2.1 Architecture and Wiring Overview

The wake word system follows a multi-layer audio pipeline. The AudioController (a Kotlin singleton object) manages the Android AudioRecord instance and captures raw PCM audio in 1024-sample chunks at 16kHz mono. These chunks are emitted through a MutableSharedFlow called wakeWordAudioFlow. The WakeWordEngine collects these chunks into a window buffer and, once the buffer reaches 8000 samples, feeds the accumulated data into a MediaPipe AudioClassifier for inference. When a classification result matches one of the target labels ("hey_jarvis", "jarvis", or "hey jarvis") with a confidence score at or above 0.50, the engine emits a signal through a SharedFlow called wakeWordDetected. The JarvisService collects this signal and transitions from IDLE to LISTENING state, then stops the wake word engine and initiates a Gemini Live WebSocket session.

The wiring between these components is generally correct in terms of coroutine flow connections. The AudioController.init() is called in JarvisService.onCreate(), which starts observing the AssistantStateManager.appState flow to automatically start and stop recording. The state transitions from WAKE_LISTENING to ACTIVE_CONVERSATION correctly route audio chunks to either the wake word flow or the Gemini audio flow. However, despite this seemingly sound architecture, the system has a fundamental flaw that renders it completely non-functional, as detailed in the next section.

2.2 Critical: Wrong / Missing TFLite Model (BUG-01)

CRITICAL

The most critical issue in the entire project is that the wake word detection system uses the wrong model. The project documentation (whathavedone.md) explicitly claims: "Successfully researched and integrated an authentic Hey Jarvis Wake Word model from the open-source microWakeWord project" and "Exchanged the generic placeholder model with a targeted local model (hey_jarvis.tflite)." However, inspecting the actual assets directory reveals that no file named hey_jarvis.tflite exists. The only TFLite model present is speech_commands.tflite (3.6 MB), which is the generic Google YAMNet-based speech commands classifier trained to recognize 35+ keywords including "yes", "no", "up", "down", etc. It does NOT include any label for "hey_jarvis", "jarvis", or "hey jarvis".

The WakeWordEngine.findAndLoadModelAsset() method searches for models in priority order: first looking for "hey_jarvis.tflite", then "speech_commands.tflite" across three directories (root, models/, tflite/). Since hey_jarvis.tflite does not exist, it falls back to speech_commands.tflite. The classification results from this model will contain labels like "_silence_", "_unknown_", "yes", "no", etc. The target labels defined in WakeWordEngine ("hey_jarvis", "jarvis", "hey jarvis") will never match any label in the model output, meaning the wake word will NEVER be detected regardless of what the user says.

Fix Recommendation

Two options exist: (A) Train a custom TFLite audio classification model specifically for "Hey Jarvis" using transfer learning on the YAMNet architecture, and place it in the assets folder as hey_jarvis.tflite. Tools like Google Teachable Machine or the TensorFlow Lite Model Maker can accomplish this with a custom dataset of "Hey Jarvis" utterances. (B) Alternatively, modify the target labels in WakeWordEngine to match labels that actually exist in the speech_commands.tflite model (such as "yes" or "go") as a temporary workaround, and update the UI prompts accordingly.

2.3 Fake Documentation Claim (BUG-02)

CRITICAL

The whathavedone.md file contains a demonstrably false claim under Step 4 (The Wake Word Engine). It states: "Successfully researched and integrated an authentic Hey Jarvis Wake Word model from the open-source microWakeWord project" and "Exchanged the generic placeholder model with a targeted local model (hey_jarvis.tflite), ensuring offline operation." Neither statement is true. The microWakeWord project may exist as an open-source effort, but its output model was never actually added to this project. The file hey_jarvis.tflite has never existed in this codebase. This is not a minor documentation oversight; it represents a fabricated implementation claim that would mislead any developer reviewing the project history or assessing completion status.

Fix Recommendation

Update whathavedone.md to accurately reflect the current state: the project uses the generic speech_commands.tflite model as a placeholder. Either train and integrate the actual Hey Jarvis model, or clearly document that the wake-word model is a TODO item that requires a custom training pipeline.

2.4 Label Mismatch: Space vs Underscore (BUG-03)

HIGH

Even if a correct model were provided, the targetLabels set in WakeWordEngine contains a label with a space character: "hey jarvis" (line 25). MediaPipe AudioClassifier models based on YAMNet use underscored labels (e.g., "hey_jarvis") without spaces. The classification result processing code lowercases and trims the label, but the space in "hey jarvis" would need to match a model output label that contains a space, which is extremely unlikely with standard YAMNet models. This means even if the model output contained "hey jarvis", the contains() check might still work, but it represents an inconsistency that could cause silent failures with different model formats.

Fix Recommendation

Remove "hey jarvis" (with space) from targetLabels and keep only the underscored variants: "hey_jarvis" and "jarvis". Alternatively, normalize both the model output labels and the target labels by replacing spaces with underscores before comparison.

2.5 Audio Buffer Size Mismatch (BUG-04)

MEDIUM

The AudioData object is created with a buffer size of 15600 samples (975ms at 16kHz), but the classification is triggered when bufferPos reaches only 8000 samples (500ms). This means only 51% of the allocated buffer is ever used before it is reset to zero. The comment on line 87 states this matches YAMNet-based models which expect 975ms windows, but the actual classification happens at 500ms, which is below the expected input size for YAMNet. This could result in degraded classification accuracy or the model rejecting the input entirely, depending on how MediaPipe handles underfilled buffers.

Fix Recommendation

Change the classification trigger threshold from 8000 to 15600 to match the AudioData buffer size and the expected YAMNet input window. This ensures the model receives a full 975ms audio window for each classification, which is the standard operating range for YAMNet-based models.

2.6 Resource Leak in loadModelFile (BUG-05)

HIGH

The loadModelFile() method in WakeWordEngine opens three resources: AssetFileDescriptor, FileInputStream, and FileChannel. On the success path, only fileDescriptor.close() and inputStream.close() are called. The FileChannel object obtained via inputStream.channel is never explicitly closed. While closing the FileInputStream may eventually close the underlying channel, this is implementation-dependent and not guaranteed by the Android API. On the exception path, none of these resources are closed at all (the catch block simply returns null), which means a failed model load attempt leaks file descriptors.

Fix Recommendation

Use a try-with-resources pattern or Kotlin use{} blocks to ensure all three resources (AssetFileDescriptor, FileInputStream, FileChannel) are properly closed in both success and exception paths. The current pattern of manually calling close() on only some resources is error-prone and should be replaced.

2.7 Missing Haptic and Audio Feedback (BUG-06)

MEDIUM

The project plan (Pillar 1, Step 4) explicitly specifies: "Play a subtle futuristic sound and trigger a haptic pulse to notify the user that JARVIS is listening." Neither of these feedback mechanisms is implemented anywhere in the codebase. When the wake word is detected, the only user-visible change is the overlay appearing on screen. There is no Vibrator service call for haptic feedback, and no MediaPlayer or SoundPool call for audio feedback. This is a significant UX gap because the user has no confirmation that the wake word was heard unless they happen to be looking at the screen.

Fix Recommendation

In the wakeWordDetected collector within JarvisService.initializeDependencies(), add a call to the Android Vibrator service to produce a short haptic pulse (e.g., 50ms). Additionally, load a short notification sound

from the raw resources folder and play it via SoundPool when the wake word is detected, before transitioning to the LISTENING state.

2.8 Wake Word System Wiring Verdict

In summary, the wake word system is **NOT properly implemented** and **NOT functionally wired**. While the code architecture (AudioController feeding WakeWordEngine feeding JarvisService) is structurally sound, the absence of a correct TFLite model means the system cannot detect the wake word under any circumstances. The wiring between components is correct in terms of coroutine flow connections, but the core inference engine receives a model that does not contain the target labels. Additionally, the buffer size mismatch (8000 vs 15600 samples) further degrades any potential classification accuracy. Until a proper "Hey Jarvis" model is trained and integrated, this system is entirely non-functional.

Chapter 3: Voice System Analysis

The voice conversation system is implemented in GeminiLiveClient.kt and handles real-time bidirectional audio streaming with Google Gemini Live API over WebSocket. This chapter evaluates whether the voice system is fully functional, properly wired, and complete according to the project plan.

3.1 Functional Components Assessment

The GeminiLiveClient implements several well-designed subsystems. The WebSocket connection management uses OkHttp with a 15-second ping interval and 12-second connect timeout, which are reasonable defaults. The reconnection logic implements exponential backoff (1s, 2s, 4s, 8s, 16s capped) with a maximum of 5 attempts, which prevents infinite reconnection loops. The audio playback system uses a bounded Channel with DROP_OLDEST overflow strategy (capacity 64 chunks, approximately 1.5 seconds of audio), which prevents unbounded memory growth during network latency spikes. The state machine is comprehensive, covering IDLE, CONNECTING, CONNECTED, LISTENING, SPEAKING, RECONNECTING, ERROR, and DISCONNECTED states.

The audio input pipeline correctly uses the AudioController.geminiAudioFlow to receive 16kHz mono PCM chunks, converts them from ShortArray to little-endian ByteArray, Base64-encodes them, wraps them in the Gemini Live API JSON format (realtimeInput.audio), and sends them over the WebSocket. The audio output pipeline receives Base64-encoded PCM data from the server, decodes it, and plays it through an AudioTrack configured for 24kHz mono 16-bit PCM, which is the correct format for Gemini Live API audio output.

3.2 WebSocket Failure State Bug (BUG-07)

HIGH

In the WebSocket onFailure() callback (GeminiLiveClient.kt line 144-155), when isClosedManually is true, the code sets the state to ERROR instead of DISCONNECTED. Compare this with the onClosed() callback

(line 131-141) which correctly checks isClosedManually and sets DISCONNECTED for manual closes. This inconsistency means that if a network error occurs during a manual disconnect (race condition between close() and onFailure()), the state will be ERROR instead of DISCONNECTED. The JarvisService responds to ERROR by showing an error overlay and eventually returning to wake mode, whereas DISCONNECTED triggers a clean return to wake mode. While the end result is similar, the ERROR path shows an error message to the user and waits 3.5 seconds, creating unnecessary confusion.

Fix Recommendation

In the onFailure() callback, check isClosedManually before setting the state, and set DISCONNECTED instead of ERROR when the connection was intentionally closed.

3.3 Thread-Safety: consecutiveSendFailures (BUG-08)

HIGH

The variable consecutiveSendFailures in GeminiLiveClient.startAudioIO() (line 341) is a regular Kotlin Int accessed from a coroutine launched on Dispatchers.IO. However, the connect() method can be called from a different coroutine context (Dispatchers.Main in JarvisService). If connect() triggers a reconnect while the audio recording coroutine is still running and modifying consecutiveSendFailures, there is a potential data race. While Kotlin coroutines are structured-concurrency safe within a single thread, the interleaving of reads and writes to a non-atomic variable across multiple suspend points can lead to lost updates.

Fix Recommendation

Replace the regular Int with java.util.concurrent.atomic.AtomicInteger for consecutiveSendFailures, similar to how reconnectAttempts and isConnecting are already implemented using atomic types.

3.4 AudioTrack Undrained on Stop (BUG-09)

MEDIUM

When stopAudioIO() is called (line 382-398), the AudioTrack is immediately stopped and released. Any PCM data that has been written to the AudioTrack internal buffer but not yet played is silently discarded. This causes audible audio cut-off at the end of Gemini responses. For example, if JARVIS is speaking a sentence and the session times out or the user dismisses the overlay, the last 100-200ms of audio will be cut off abruptly, creating an unpleasant user experience.

Fix Recommendation

Before calling audioTrack.stop(), wait for the playback head position to reach the total written data. Alternatively, use audioTrack.setNotificationMarkerPosition() to detect when all buffered audio has been played before stopping the track.

3.5 Missing AEC and VAD (BUG-10, BUG-11)

HIGH

The project plan (Pillar 2, Step 3) explicitly requires two critical audio processing features: Acoustic Echo Cancellation (AEC) and Voice Activity Detection (VAD). AEC is needed so that when JARVIS speaks through the speaker, the microphone does not pick up its own audio and send it back to the Gemini API as user input. Without AEC, the system will echo-loop: JARVIS speaks, mic picks it up, Gemini interprets it as user speech, generating a response, which plays through the speaker again, creating an infinite feedback loop. VAD is needed to detect when the user is speaking versus when there is silence, enabling the barge-in feature (interrupting JARVIS mid-sentence). Neither AEC nor VAD is implemented anywhere in the codebase. The AudioController captures raw PCM audio without any processing, and the GeminiLiveClient sends all audio directly to the API without local filtering.

Fix Recommendation

For AEC: Use Android built-in AcousticEchoCanceler (available on API level 16+) by applying it as an audio effect on the AudioRecord instance in AudioController. Check availability with AcousticEchoCanceler.isAvailable() before applying. For VAD: Integrate the WebRTC VAD library (available as a native dependency) to classify audio frames as speech or non-speech. Only send audio to the Gemini API when VAD indicates speech is detected.

3.6 Hardcoded Model Name (BUG-12)

MEDIUM

The Gemini model name "models/gemini-3.1-flash-live-preview" is hardcoded as a default parameter in the GeminiLiveClient constructor (line 49). This is a preview/unstable model identifier that may be deprecated or changed by Google at any time. There is no way for the user to select a different model through the settings screen, and no fallback mechanism if this model becomes unavailable. The validation test in SettingsViewModel tests against the Gemini models list endpoint but does not verify that this specific live model is accessible.

Fix Recommendation

Make the model name configurable through SettingsRepository, similar to how the API key is stored. Provide a dropdown or text field in SettingsScreen for model selection. Use a stable model name as the default and allow the preview model as an advanced option.

3.7 API Key Exposure in URL (BUG-13)

HIGH

The Gemini API key is passed as a URL query parameter in the WebSocket URL on line 116 of GeminiLiveClient.kt. While this is the documented Google API approach for WebSocket connections, URL query parameters are logged in proxy servers, CDN logs, and Android system logs. This represents a security concern for the API key, especially on enterprise or school networks with mandatory monitoring. The Google Gemini Live API does not currently support header-based authentication for WebSocket connections, so this

is partly an API design limitation, but the risk should be documented.

Fix Recommendation

While this cannot be fully mitigated due to Google API design, add a comment documenting the security implication. Consider rotating the API key periodically and monitoring Google Cloud Console for unusual usage patterns. If Google adds header-based authentication support in the future, migrate to it.

3.8 Voice System Wiring Verdict

The voice system is **partially functional**. The WebSocket connection, audio streaming, playback, reconnection logic, and state management are well-implemented and would work correctly with a valid API key and network connection. However, the absence of AEC means the system will suffer from echo feedback when the device speaker is used (though it may work acceptably with wired or Bluetooth headsets). The absence of VAD means the barge-in feature does not work, and all audio (including silence) is streamed to the API, wasting bandwidth. The system is properly wired from AudioController through GeminiLiveClient to the WebSocket, and the state flow is correctly propagated back to the JarvisService for UI updates.

Chapter 4: Fake, Stub, and Missing Code

4.1 Dead Code: applet/ Directory (BUG-14)

MEDIUM

The project contains an applet/ directory (app/applet/) with two Kotlin files: a SettingsScreen.kt and a SettingsViewModel.kt. These files implement a completely different (and older/simpler) version of the settings UI compared to the actual settings screen used by the app. The applet SettingsScreen takes a NavController parameter (implying it was designed for a different navigation architecture), uses collectAsState instead of collectAsStateWithLifecycle, lacks the API key testing feature, and has no permission management section. The applet SettingsViewModel uses a different SharedPreferences file name ("secret_prefs" vs "jarvis_secure_prefs"), does not have validation state tracking, and directly creates the MasterKey in the constructor instead of using lazy initialization. The settings.gradle.kts only includes ":app", so this directory is not compiled. This is dead code that should be removed entirely.

4.2 Unused ChatMessage Data Class (BUG-15)

LOW

ChatMessage.kt defines a data class with fields for id, text, isUser, and isSystemMessage. However, this class is never instantiated or referenced anywhere in the active codebase. No chat history is displayed or stored. The Gemini Live conversation is purely real-time with no persistence. This is either leftover code from a planned chat history feature or a stub that was never implemented.

4.3 Entire Pillar 3 Missing: Proactive Intelligence (BUG-16)

CRITICAL

The project plan describes three pillars, with Pillar 3 (Proactive Intelligence System) being one of the core differentiators. This pillar was supposed to include: (1) a WorkManager Heartbeat Worker that periodically fetches device usage stats, (2) a Room Database with AppUsageTable and EventTable for storing usage patterns, (3) a Pattern Detection Algorithm (FP-Growth or similar) for finding correlations in user behavior, (4) a Proactive Scorer that evaluates context before initiating conversations, and (5) a Proactive Wake mechanism that automatically enters AWAKE state when patterns are detected. None of these components exist in the codebase. There is no WorkManager worker, no Room database, no DAO, no Entity classes, no UsageStatsManager integration, no pattern detection logic, and no proactive scoring.

The libs.versions.toml defines Room dependencies (room-runtime, room-ktx, room-compiler at version 2.7.0) but they are not declared in build.gradle.kts dependencies. The AndroidManifest.xml declares the PACKAGE_USAGE_STATS permission, and PermissionsHelper has methods for checking and requesting Usage Access permission, but these methods are never called from any screen or service. This represents approximately 40-50% of the planned feature set that is completely absent from the codebase.

4.4 Empty ProGuard Rules (BUG-17)

MEDIUM

The proguard-rules.pro file contains only the default Android Studio template comments with no actual rules. Since isMinifyEnabled is false in the release build type, this does not currently cause build failures. However, when minification is eventually enabled (which is essential for release builds to reduce APK size and obfuscate code), the lack of ProGuard rules for OkHttp, MediaPipe, security-crypto, and other libraries will cause runtime crashes due to class and method obfuscation. Libraries like OkHttp require specific keep rules for their reflection-based JSON parsing and interceptor chains.

4.5 Stub Test Files (BUG-18)

LOW

All three test files are stubs with no actual test coverage. ExampleUnitTest.kt contains a single test that verifies 2+2=4. ExampleRobolectricTest.kt likely contains similar placeholder tests. GreetingScreenshotTest.kt references a screenshots/greeting.png file, but there is no test logic that exercises any of the application code. There are no tests for WakeWordEngine, AudioController, GeminiLiveClient, SettingsRepository, or any other component. Given the complexity of the audio pipeline, WebSocket management, and state machine, the absence of unit tests is a significant quality gap.

4.6 .aistudio Directory Stub (BUG-19)

LOW

The assets/.aistudio/ directory contains only a .gitignore file. This appears to be a leftover from the Google AI Studio project template. It serves no functional purpose in the application and should be removed.

Chapter 5: Hardcoded Values and Configuration Issues

Multiple critical parameters are hardcoded throughout the codebase rather than being configurable through settings or resource files. This section catalogs each hardcoded value and explains why it should be externalized for production readiness.

ID	Severity	Issue	File
BUG-20	MEDIUM	**System prompt hardcoded in sendSetupMessage()** The system instruction is hardcoded. Users cannot customize JARVIS personality.	`GeminiLiveClient.kt:188`
BUG-21	MEDIUM	**Wake threshold 0.50f hardcoded** Different environments need different thresholds for reliable detection.	`WakeWordEngine.kt:23`
BUG-22	MEDIUM	**Session timeout 120s hardcoded** Users may want longer or shorter conversation sessions.	`JarvisService.kt:85`
BUG-23	LOW	**Cooldown 2000ms hardcoded** Prevents rapid re-triggering but may not suit all users.	`WakeWordEngine.kt:24`
BUG-24	LOW	**Audio buffer sizes hardcoded** Multiple buffer sizes (1024, 15600, 8000) hardcoded across files.	`AudioController.kt`, `WakeWordEngine.kt`
BUG-25	LOW	**Sample rates hardcoded** Input (16kHz) and output (24kHz) hardcoded in multiple locations.	`AudioController.kt:65`, `GeminiLiveClient.kt:274`

Chapter 6: Architecture and Wiring Issues

6.1 Dual State Management Desynchronization (BUG-26)

CRITICAL

The application maintains two independent state management systems that track overlapping but not identical state: (1) JarvisServiceState, a companion object with MutableStateFlows for isRunning, assistantState (using AssistantState enum), and lastError; and (2) AssistantStateManager, a companion object with a MutableStateFlow for appState (using AppState enum). Both are static singletons. The JarvisService updates both on every state transition, but they use different enum types with different granularities. AssistantState has 5 values (IDLE, LISTENING, UNDERSTANDING, SPEAKING, ERROR) while AppState has 3 values (IDLE, WAKE_LISTENING, ACTIVE_CONVERSATION). The AudioController only observes AppState to decide where to route audio chunks, while the UI only observes AssistantState for display. If any code path updates one without the other, the audio routing and UI display will desynchronize, causing silent failures

where audio is routed incorrectly or the UI shows the wrong state.

Fix Recommendation

Consolidate into a single state management system. Either extend AppState to include the UI-relevant states (LISTENING, UNDERSTANDING, SPEAKING) and have the UI derive its display state from AppState, or remove AssistantStateManager entirely and have AudioController observe JarvisServiceState. The key principle is that there should be a single source of truth for the application state.

6.2 AudioController Race Condition (BUG-27)

HIGH

The AudioController singleton manages audio recording based on AssistantStateManager.appState changes. When the state changes from WAKE_LISTENING to ACTIVE_CONVERSATION, the current recording coroutine continues running while the state transition happens. During this brief window, the recording coroutine may still emit audio chunks to the old flow (wakeWordAudioFlow) because it reads the state value at the point of emission. This results in a small number of audio chunks being routed to the wrong destination during state transitions, potentially causing the wake word engine to receive Gemini-bound audio or vice versa.

Fix Recommendation

Cancel the current recordingJob before starting a new one when the state changes. In startRecording(), always cancel any existing recordingJob first. Alternatively, use a Mutex to ensure that only one recording coroutine is active at any time.

6.3 Unused Dependencies in libs.versions.toml (BUG-28)

MEDIUM

The libs.versions.toml defines numerous dependencies that are never used in the actual build.gradle.kts: Room (room-runtime, room-ktx, room-compiler), CameraX (camera-camera2, camera-lifecycle, camera-view, camera-core), Firebase (firebase-bom, firebase-ai, firebase-appcheck-recaptcha, firebase-firestore, firebase-auth), Credentials (credentials, credentials-play-services, googleid), Retrofit (retrofit, converter-moshi), Moshi (moshi-kotlin, moshi-kotlin-codegen), Coil (coil-compose), Play Services Location, and more. These inflate the version catalog and create maintenance overhead. Additionally, the tensorflow-lite-task-audio entry incorrectly points to the tensorflow-lite-support version number, which is a version mismatch bug.

6.4 OkHttp Version Mismatch (BUG-29)

MEDIUM

The libs.versions.toml defines okhttp version as "4.10.0" (line 33), but the build.gradle.kts hardcodes "com.squareup.okhttp3:okhttp:4.12.0" (line 79). The version catalog entry is never referenced. This means the

version catalog and the actual dependency are out of sync. If someone updates the version catalog expecting it to take effect, the hardcoded version in build.gradle.kts will silently override it.

Fix Recommendation

Replace the hardcoded dependency in build.gradle.kts with a version catalog reference: implementation(libs.okhttp). Update the version in libs.versions.toml to 4.12.0 for consistency.

6.5 Accompanist Permissions Unused (BUG-30)

LOW

The accompanist-permissions library (version 0.37.3) is declared as a dependency in build.gradle.kts but is never used anywhere in the code. The PermissionsHelper utility uses ContextCompat directly for permission checks, and the permission request flow uses ActivityResultContracts directly in Composable functions. This unused dependency adds approximately 50KB to the APK size unnecessarily.

6.6 No Code Minification in Release (BUG-31)

MEDIUM

The release build type has isMinifyEnabled = false, meaning no R8/ProGuard optimization is applied. For a production application, this results in a significantly larger APK, no code obfuscation (making it easier to reverse-engineer), and no dead code elimination. Combined with the empty proguard-rules.pro, this means the release build is essentially an unoptimized debug build with a release signing key.

6.7 Lint Checks Disabled (BUG-32)

MEDIUM

The build.gradle.kts sets both abortOnError = false and checkReleaseBuilds = false for lint. This means all lint warnings and errors are silently ignored during both debug and release builds. While some lint warnings can be noisy, completely disabling lint means missing real issues like missing translations, unused resources, incorrect API usage, accessibility problems, and performance issues.

6.8 Contacts Permission Declared But Unused (BUG-33)

MEDIUM

The AndroidManifest.xml declares READ_CONTACTS permission, the PermissionsScreen requests it, and the PermissionsHelper has a method to check it. However, no code in the application ever reads contacts data. The plan mentions voice calling features but no telephony/Dialer API integration exists. Requesting a sensitive permission like READ_CONTACTS without using it is a privacy concern and creates unnecessary friction in the onboarding flow.

6.9 Kotlin Android Plugin Missing (BUG-34)

LOW

The build.gradle.kts applies only the kotlin-compose plugin and the android.application plugin. There is no explicit kotlin-android plugin declaration. In modern AGP (9.1.1 as defined in libs.versions.toml), the kotlin-android functionality may be pulled in by kotlin-compose, but this is an implicit dependency that could break with future AGP versions. The standard practice is to explicitly declare both plugins.

Chapter 7: UI/UX Issues

7.1 Overlay Error State Not Cleared on Dismiss (BUG-35)

HIGH

When the user taps the dismiss/close button on the JarvisOverlay during an error state, the onDismiss callback calls startWakeWordMode() in JarvisService. This correctly transitions the audio state back to WAKE_LISTENING and the assistant state to IDLE. However, the error message stored in JarvisServiceState.lastError is never cleared. This means the error banner on the main screen (MainActivity) will continue displaying the old error message even after the user has dismissed the overlay and the service has returned to normal wake-word listening mode. The only way to clear it is to stop and restart the service entirely, which is a poor user experience.

Fix Recommendation

In the onDismiss handler (or in startWakeWordMode()), call JarvisServiceState.clearError() to reset the error message when transitioning back to wake word mode.

7.2 Permissions Screen Allows Skip Without Core Permissions (BUG-36)

MEDIUM

The PermissionsScreen has a "SKIP FOR NOW" button that navigates to the home screen regardless of whether core permissions (Microphone and Overlay) are granted. The main button also navigates forward even when core permissions are not granted, only showing a Toast message. This means users can reach the home screen and attempt to start the service without the required permissions, leading to a poor experience where the service fails silently or shows error messages that the user may not understand.

7.3 Permissions Count Misleading (BUG-37)

LOW

The SettingsScreen shows "X of 3 permissions granted (Microphone, Overlay, Notifications)" but only Microphone and Overlay are actually required for core functionality. Notification permission is optional. The count implies all three are equally important, which may confuse users who see "2 of 3" and think they are missing a critical permission when in fact they have everything needed.

7.4 Architecture Status Card Shows Misleading Info (BUG-38)

MEDIUM

The home screen Architecture Feature Row for the wake-word listener shows isActive = hasAudio (microphone permission). This is misleading because having microphone permission does not mean the wake-word listener is actually working. The listener also requires a valid TFLite model (which is the wrong model, as discussed in BUG-01) and the service to be running. Similarly, the Gemini row shows isActive = apiKey.isNotBlank(), which only indicates the key is entered, not that the WebSocket connection works. These status indicators give users a false sense of system completeness.

7.5 Duplicate SettingsRepository Instances (BUG-39)

MEDIUM

The SettingsRepository is instantiated directly in multiple places using remember { SettingsRepository(context) }: in MainActivity (line 140), in SettingsScreen (line 58), and in JarvisService (line 150). Each instance creates its own lazy EncryptedSharedPreferences, which means the _apiKeyFlow is not shared between instances. When the user saves a new API key in SettingsScreen, the MainActivity instance may not reflect the update because it has a separate MutableStateFlow. The SettingsViewModel works around this by using stateIn() with WhileSubscribed, but the direct SettingsRepository usage in MainActivity and JarvisService creates independent state that can go stale.

Fix Recommendation

Convert SettingsRepository to a singleton object (similar to AudioController and AssistantStateManager) or inject it via a dependency injection framework like Hilt. This ensures all consumers share the same SharedPreferences instance and observe the same StateFlow.

7.6 Service Running State Not Synced (BUG-40)

HIGH

JarvisServiceState.isRunning is set to true in onCreate() and false in onDestroy(). However, if the service crashes (uncaught exception in the service scope), onDestroy() may not be called, leaving isRunning permanently set to true. The main screen would then show "SYSTEM ONLINE" even though the service is dead. There is no mechanism to detect service crashes and reset the state. Additionally, if the user force-stops the app from system settings, the static state in JarvisServiceState is reset (process death), which is correct, but if the service is killed by the OS for resource reasons, the state may not be properly cleaned up.

Fix Recommendation

Use Android Service connection binding in the Activity to get reliable lifecycle callbacks. Alternatively, periodically check if the service is actually running using ActivityManager.getRunningServices() or by sending a ping broadcast that the service responds to.

7.7 Emoji in Overlay Code (BUG-41)

LOW

The JarvisOverlay.kt uses emoji characters rendered as Text composables. While modern Android devices generally support emoji rendering, the appearance of these emoji varies significantly across Android versions, OEM skins (Samsung One UI, Xiaomi MIUI), and installed font packs. On some devices, the emoji may render as black-and-white outlines, empty boxes, or not render at all. For a consistent UI experience, vector icons or Compose Icons should be used instead of emoji characters in production code.

Chapter 8: Consolidated Bug Summary

The following table provides a complete reference of all 41 issues identified in this audit, organized by severity. Each issue is cross-referenced with the detailed analysis in the preceding chapters.

ID	Severity	Issue	File
BUG-01	CRITICAL	**Wrong TFLite model (speech_commands instead of hey_jarvis)** Wake word NEVER triggers	`WakeWordEngine.kt, assets/`
BUG-02	CRITICAL	**Fake documentation claim about hey_jarvis.tflite** Fabricated completion claim	`whathavedone.md`
BUG-03	HIGH	**Label mismatch: space vs underscore in targetLabels** 'hey jarvis' never matches	`WakeWordEngine.kt:25`
BUG-04	MEDIUM	**Audio buffer size mismatch (8000 vs 15600)** Partial data classified	`WakeWordEngine.kt:157`
BUG-05	HIGH	**Resource leak in loadModelFile** FileChannel not closed	`WakeWordEngine.kt:222-236`
BUG-06	MEDIUM	**Missing haptic/audio feedback on wake detection** No user confirmation	`JarvisService.kt`
BUG-07	HIGH	**WebSocket onFailure sets wrong state on manual close** ERROR vs DISCONNECTED	`GeminiLiveClient.kt:144-155`
BUG-08	HIGH	**Thread-unsafe consecutiveSendFailures** Data race risk	`GeminiLiveClient.kt:341`
BUG-09	MEDIUM	**AudioTrack undrained on stop** Audio cutoff at end	`GeminiLiveClient.kt:382-398`
BUG-10	HIGH	**Missing Acoustic Echo Cancellation** Echo feedback loops	`AudioController.kt`
BUG-11	HIGH	**Missing Voice Activity Detection** No barge-in support	`AudioController.kt`
BUG-12	MEDIUM	**Hardcoded unstable Gemini model name** May break without notice	`GeminiLiveClient.kt:49`
BUG-13	HIGH	**API key exposed in WebSocket URL** Logged in proxies	`GeminiLiveClient.kt:116`

ID	Severity	Issue	File
BUG-14	MEDIUM	**Dead applet/ directory with old code** Unmaintained duplicates	`app/applet/`
BUG-15	LOW	**Unused ChatMessage data class** Dead code	`ChatMessage.kt`
BUG-16	CRITICAL	**Entire Pillar 3 (Proactive Intelligence) missing** 40-50% of planned features absent	`N/A`
BUG-17	MEDIUM	**Empty ProGuard rules file** Will crash when minification enabled	`proguard-rules.pro`
BUG-18	LOW	**Stub test files with no real coverage** No quality assurance	`app/src/test/`
BUG-19	LOW	**Empty .aistudio/ directory stub** Template leftover	`assets/.aistudio/`
BUG-20	MEDIUM	**System prompt hardcoded** Cannot customize personality	`GeminiLiveClient.kt:188`
BUG-21	MEDIUM	**Wake threshold 0.50f hardcoded** Different envs need different values	`WakeWordEngine.kt:23`
BUG-22	MEDIUM	**Session timeout 120s hardcoded** Inflexible for users	`JarvisService.kt:85`
BUG-23	LOW	**Cooldown 2000ms hardcoded** May not suit all users	`WakeWordEngine.kt:24`
BUG-24	LOW	**Audio buffer sizes hardcoded** Scattered magic numbers	`AudioController.kt`
BUG-25	LOW	**Sample rates hardcoded** Inconsistent definitions	`GeminiLiveClient.kt`
BUG-26	CRITICAL	**Dual state management desync** State can diverge	`JarvisServiceState + AssistantStateManager`
BUG-27	HIGH	**AudioController race on state transition** Audio routed to wrong destination	`AudioController.kt:37-44`
BUG-28	MEDIUM	**Unused dependencies in version catalog** Maintenance overhead	`libs.versions.toml`
BUG-29	MEDIUM	**OkHttp version mismatch (4.10.0 vs 4.12.0)** Silent version override	`build.gradle.kts:79`
BUG-30	LOW	**Accompanist Permissions unused** Unnecessary 50KB	`build.gradle.kts:66`
BUG-31	MEDIUM	**No code minification in release** Large APK, no obfuscation	`app/build.gradle.kts:25`
BUG-32	MEDIUM	**Lint checks completely disabled** Issues go undetected	`app/build.gradle.kts:39-40`
BUG-33	MEDIUM	**READ_CONTACTS declared but unused** Privacy concern	`AndroidManifest.xml:27`

ID	Severity	Issue	File
BUG-34	LOW	**Kotlin android plugin not explicit** Fragile implicit dep	app/build.gradle.kts:3-4
BUG-35	HIGH	**Overlay error not cleared on dismiss** Stale error banner	JarvisService.kt:317
BUG-36	MEDIUM	**Permissions skip allows no-permission access** Service fails silently	PermissionsScreen.kt
BUG-37	LOW	**Permissions count includes optional notif** Misleading 2 of 3 count	SettingsScreen.kt:390
BUG-38	MEDIUM	**Architecture status card misleading** False system ready	MainActivity.kt:478-493
BUG-39	MEDIUM	**Duplicate SettingsRepository instances** Stale state across screens	MainActivity, Settings, Service
BUG-40	HIGH	**Service running state not synced on crash** Ghost ONLINE status	JarvisServiceState.kt
BUG-41	LOW	**Emoji in overlay may render inconsistently** Varies by device/OEM	JarvisOverlay.kt:110-111

Chapter 9: Priority Recommendations

9.1 Immediate (Must Fix Before Any Testing)

The following issues must be resolved before the application can be tested end-to-end. These represent fundamental flaws that prevent the core functionality from working at all. First, train and integrate a custom "Hey Jarvis" TFLite audio classification model (BUG-01). Without this, the entire wake word system is non-functional and no user interaction is possible. Second, consolidate the dual state management into a single source of truth (BUG-26). The current two-system approach is fragile and will cause desynchronization bugs under real-world usage. Third, fix the AudioController race condition by ensuring clean recording job lifecycle management during state transitions (BUG-27). Fourth, correct the fake documentation in whathavedone.md to accurately reflect the current implementation state (BUG-02).

9.2 Short-Term (Fix Before Production Release)

These issues should be addressed before any production or beta release. Implement Acoustic Echo Cancellation using Android AcousticEchoCanceler to prevent echo feedback loops (BUG-10). Fix the WebSocket onFailure state handling to properly detect manual disconnects (BUG-07). Make consecutiveSendFailures thread-safe with AtomicInteger (BUG-08). Fix the resource leak in loadModelFile by using use{} blocks (BUG-05). Clear error state when dismissing the overlay (BUG-35). Sync service running state with actual service lifecycle (BUG-40). Remove dead code (applet directory) and unused dependencies (BUG-14, BUG-28). Enable code minification with proper ProGuard rules for release builds

(BUG-17, BUG-31).

9.3 Medium-Term (Architecture Improvements)

These improvements will enhance the robustness and maintainability of the application. Convert SettingsRepository to a singleton to prevent stale state (BUG-39). Make all hardcoded parameters configurable through settings (BUG-20 through BUG-25). Implement Voice Activity Detection for proper barge-in support (BUG-11). Add haptic and audio feedback for wake word detection (BUG-06). Fix the audio buffer size mismatch to use the full YAMNet window (BUG-04). Add comprehensive unit tests for all core components (BUG-18). Enable lint checks for release builds (BUG-32).

9.4 Long-Term (Feature Completion)

These items address the gap between the current implementation and the planned three-pillar architecture. Implement the entire Pillar 3 Proactive Intelligence System including Room Database, WorkManager scheduling, UsageStatsManager integration, and pattern detection (BUG-16). Add contact-based voice calling features or remove the READ_CONTACTS permission (BUG-33). Make the Gemini model name and system prompt user-configurable (BUG-12, BUG-20). Implement proper audio drain on stop to prevent cutoff (BUG-09). Consider replacing emoji in the overlay with vector icons for consistent cross-device rendering (BUG-41).